

Hybrid Profanity Detection and Review Rewriting System

Overview

This module implements a **two-stage hybrid inference pipeline** for customer review moderation using a **LoRA-fine-tuned Large Language Model (LLM)**.

The system performs:

- 1. **Content Analysis (Classification)**
 - Detects **profanity**
 - Determines **sentiment** (positive / negative / neutral)
- 2. **Conditional Text Rewriting (Generation)**
 - Removes profanity **only when necessary**
 - Preserves the **original intent, tone, and sentiment**
 - Uses **different rewriting strategies** for positive and negative reviews

The key design choice is **adapter switching**:

- **LoRA adapter enabled** → structured moderation analysis
- **LoRA adapter disabled** → fluent, unbiased rewriting using the base model

Architecture Summary

Stage	Model State	Purpose
Stage 1	Base model + LoRA adapter	Profanity & sentiment detection
Stage 2	Base model only (adapter off)	Natural-language rewriting

This separation prevents **classification bias from leaking into generation**, a common failure mode in moderation systems.

Model Components

Base Model

- **Model:** LiquidAI/LFM2.5-1.2B-Instruct
- **Type:** Instruction-tuned causal language model
- **Precision:** FP16
- **Device Mapping:** Automatic (device_map="auto")

Adapter

- **Technique:** LoRA (Low-Rank Adaptation)
- **Purpose:** Specialized content moderation behavior
- **Loaded via:** `peft.PeftModel`

Code Walkthrough

1. Environment and Paths

```
adapter_path = "/content/drive/MyDrive/lfm-profanity-lora"  
base_model_name = "LiquidAI/LFM2.5-1.2B-Instruct"
```

- `adapter_path` must contain:
 - LoRA adapter weights
 - Tokenizer files
- `local_files_only=True` enforces offline loading

2. Tokenizer Loading

```
tokenizer = AutoTokenizer.from_pretrained(adapter_path, local_files_only=True)
```

Important design decision:

The tokenizer is loaded from the adapter directory to guarantee **vocabulary alignment** with the fine-tuned model.

3. Base Model Loading

```
model = AutoModelForCausalLM.from_pretrained(  
    base_model_name,  
    dtype=torch.float16,  
    device_map="auto",  
    trust_remote_code=True,  
)
```

- FP16 reduces memory footprint
- `trust_remote_code=True` is required by LiquidAI models
- GPU is automatically selected when available

4. Adapter Injection

```
model = PeftModel.from_pretrained(model, adapter_path, local_files_only=True)
```

At this point:

- The model behaves as a **moderation classifier**
- Generation is constrained and structured

Core Function: `hybrid_analyze_review`

This function encapsulates the entire pipeline.

Stage 1: Profanity & Sentiment Detection

Prompt Strategy

The system uses a **strictly formatted instruction prompt**:

- Requests:
 1. Profanity presence (yes/no)
 2. Sentiment classification

This guarantees **machine-parsable output**, rather than free-form text.

Generation Parameters

```
max_new_tokens=64  
temperature=0.1
```

Low temperature ensures:

- Deterministic
- Repeatable
- Low hallucination risk

Output Parsing

```
has_profanity = "Profanity: yes" in analysis  
is_positive = "Sentiment: positive" in analysis.lower()
```

Criticism:

- This string-matching approach is fragile
- A structured format (JSON / XML) would be more robust

Stage 2: Conditional Rewrite Logic

Rewriting is triggered **only if profanity is detected**.

Adapter Deactivation

```
with model.disable_adapter():
```

This is a critical design choice:

- Prevents classification bias
- Restores full generative capacity
- Improves linguistic naturalness

Rewrite Strategy Matrix

Profanity	Sentiment	Action
No	Any	No rewrite
Yes	Positive	Enthusiasm-preserving rewrite
Yes	Negative	Minimal, factual rewrite

Positive Rewrite Strategy

Objective:

Remove slang/profanity **without weakening praise**

Key Constraints:

- Preserve excitement
- No sentiment dilution
- Professional but energetic tone

Higher temperature:

```
temperature = 0.6
```

This allows expressive paraphrasing.

Negative Rewrite Strategy

Objective:

Remove insults while preserving **complaint fidelity**

Key Constraints:

- Minimal lexical substitution
- Preserve:
 - Timelines
 - Failures
 - Monetary impact
 - Accountability

Lower temperature:

```
temperature = 0.3
```

This minimizes unintended semantic drift.

Prompt Engineering Pattern

Each rewrite prompt contains:

1. System role definition
2. Explicit transformation rules
3. Few-shot examples
4. The user review

This enforces **behavioral anchoring**, reducing variance.

Output Cleanup

```
rewrite = rewrite.split("assistant")[-1].strip()
rewrite = rewrite.replace("Rewritten:", "").strip()
rewrite = rewrite.splitlines()[0]
```

This post-processing ensures:

- No role leakage
- Single-line output
- Clean downstream consumption

Batch Testing

The script includes **four test categories**:

1. Profane + Negative
2. Non-profane + Negative

3. Profane + Positive

4. Non-profane + Positive

This validates:

- False-positive avoidance
- Sentiment preservation
- Rewrite correctness

Usage Instructions

Prerequisites

- CUDA-capable GPU
- PyTorch with CUDA support
- Installed dependencies:
 - `transformers`
 - `peft`
 - `torch`

Running the Script

```
python inference.py
```

Programmatic Usage

```
result = hybrid_analyze_review("This product is fucking amazing!")  
print(result)
```

Design Strengths

- Clean separation of **analysis vs generation**
- Correct use of **adapter disabling**
- Thoughtful temperature control
- Strong prompt constraints
- Realistic moderation behavior